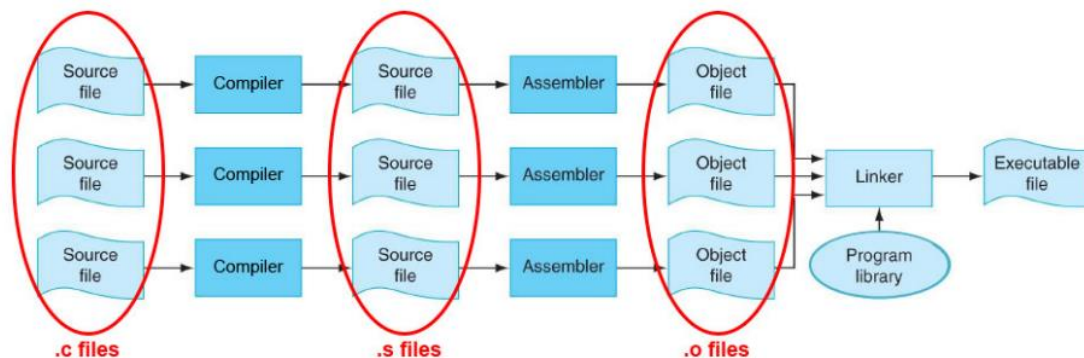


Tutorial 8: Program Execution and Instruction Execution

Q1. Explain what are Interpreter and Compiler in the context of computer architectural program execution. Give example situation when to use Interpreter or Compiler.

Interpreter	Compiler
Directly executes a program in the source language. (translate one instruction at a time)	Converts a program from the source language to an equivalent program in another language. (translate whole code into machine code)
Generally interpreter is used to interpret high level language when efficiency is not critical. (no optimization, need all of source code available)	Compiler is typically used to translate high level language code to low level language when higher performance is required. (can optimize code, single file to execute)

Q2. Elaborate, with a suitable diagram, to show the overall picture of the functions of C.A.L.L. (Compiler, Assembler, Linker and Loader) in the context of translating higher level language program to suitable file to be executed in hardware.



Compiler : Translates higher level language code, eg. Intro2CoA.c to lower level language (typically assembly language) code, eg. Intro2CoA.s.

Assembler : Translates assembly language code, eg. Intro2CoA.s into object code or known as object files, eg. Intro2CoA.o. It converts pseudo-instructions to MIPS instructions.

Linker : combine independently assembled machine language programs (eg. Intro2CoA.o, lib.o) and resolves all remaining undefined labels from the relocation information and symbol table. The output is the executable file, eg. Intro2CoA.exe.

Loader : converts executable code (eg. Intro2CoA.exe) into binary file format (Intro2CoA.bin) so can load into hardware for program execution.

Q3. Describe how Symbol Table may be useful for program translation process.
Symbol table is a table that matches the names of symbol to the addresses of the memory words that instruction occupies. It consists of Label and Data; Labels specifies target function the program is calling, whereas Data informs the constant and variables declared under .data section. This information is needed during linking process.

Q4. List the functions performed by Assembler and Loader during MIPS machine program execution.

Functions performed by MIPS machine assembler :

- Reads and uses Directives (such as .text, .data, .global sym)
- Replace psedoinstructions
- Produce machine language (creates object file)
- Creates a checklist for linker (relocation table)
- Resolves addresses by making 2 passes

Functions perform by Loader:

- Reads executable file's header to determine the size of the text and data segment
- Creates new address space for program
- Copies instruction / data from executable file into new address space
- Initializes machine registers
 - Sets \$gp to 0x1000 8000 (the middle of the global data segment)
 - Sets \$sp to 0x7FFF FFFC (the top of the dynamic data segment)
- Jumps to start-up routine
 - Perform a jal 0x0040 0000

Q5. Compare the difference between Static and Dynamic Linked libraries (SLL and DLL) during the program translation process.

In **Static Linked Libraries (SLL)**, the targeted library is now part of executable, if the library updates the user won't get automatic update but have to recompile. SLL includes the *entire* library even if not all of it will be used, and it's executable is self-contained, which mean the user only need this file to execute the program.

In **Dynamic Linked Libraries (DLL)**, the involved libraries are not part of executable but will be link when required at runtime, this means the executed program will obtain updated libraries and executed faster since only involved those related libraries but not all.

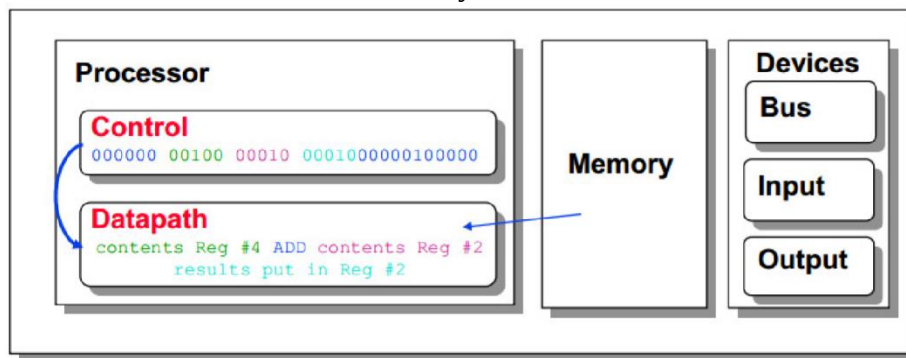
Q6. Describe the Instruction Execution process in detail, shows how information move between Memory, Control module and Datapath, using a suitable diagram.

During instruction execution cycle, operands are fetch from memory, then Datapath executes the instruction as directed by control module.

Datapath needs to have circuitry to perform following tasks:

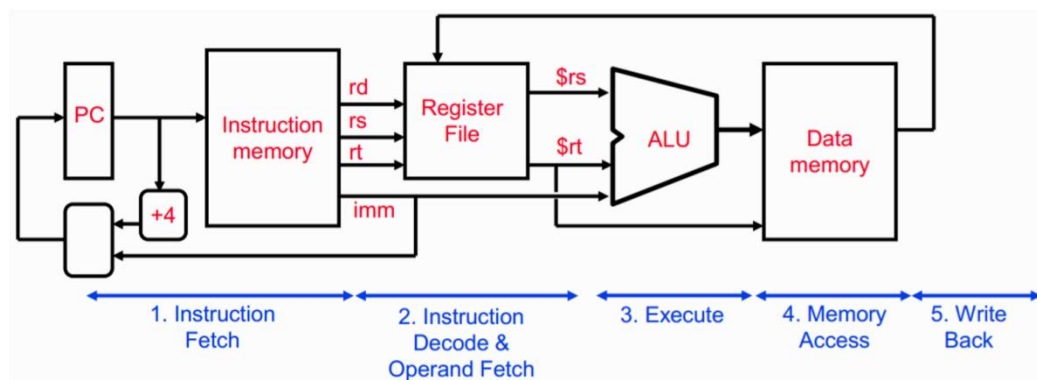
- Execute instructions – functional units (eg. adder) and storage locations (eg. register file)

- Interconnect the functional units accordingly
- Load data from and store to memory



Instruction Execution cycle process block diagram

Q7. Show the datapath abstract implementation view with respect to each instruction process stage (ie. IF, ID, Exe, Mem, WB) for the single cycle basic RISC processor core. Thus briefly describe all the five stages of instruction execution processes.



(IF) Instruction Fetch – a 32-bit instruction was fetched from the cache. The Program Counter (PC) register holds the current instruction's address to execute. At the same time, the PC predictor predicts the address of the next instruction by incrementing the PC by 4.

(ID) Instruction Decode – instruction is interpreted by control logic to determine what instruction format, followed by what instruction and which registers/operand is being used.

(Ex) Instruction Execution – Arithmetic and Logic Unit is used to perform arithmetic for the involved operands which are reside in the registers.

(Mem) Memory Access – involved operands are fetch from memory.

(WB) Write Back – the outcome of the execution is store into the register file.